MSML606 Homework 4 Report

None

Name: Amey Hengle
UID: 122283961
Email: ameyhen@umd.edu

I) Code and Results

1. Implementation

Randomized Quicksort

I implemented Randomized Quicksort as a recursive divide-and-conquer algorithm. The key idea is to select a pivot randomly from the current subarray before each partition step. I do this by swapping a randomly chosen element into the last position, then running the standard Lomuto partition around it. After partitioning, the pivot is placed in its final sorted position, and I recursively sort the left and right subarrays. Randomizing the pivot prevents worst-case behavior on already-sorted or reverse-sorted inputs, which would otherwise occur if the first or last element was always picked.

The implementation uses three functions: randomizedPartition (picks a random index and swaps it to the end), partition (the standard Lomuto partition logic), and randomizedQuicksort (the recursive driver). I also set sys.setrecursionlimit(200000) to allow sorting of large arrays without hitting Python's default cap of 1000.

Heapsort

I implemented Heapsort in two phases without using any built-in heap libraries. In the first phase, I call buildMaxHeap, which converts the input array into a max-heap by calling heapify on every non-leaf node starting from index n // 2 - 1 down to index 0. In the second phase, I repeatedly swap the root (the maximum element) with the last element in the active heap, shrink the heap boundary by one, and call heapify on the new root to restore the heap property. After n-1 such extractions the array is fully sorted in ascending order.

---

2. Time complexity analysis

Case	Randomized Quicksort	Heapsort
Best	O(n log n)	O(n log n)
Average	O(n log n)	O(n log n)
Worst	$O(n^2)$	O(n log n)

Randomized Quicksort: In the best case, each pivot splits the array into two equal halves, giving a recursion depth of log n with O(n) work per level, totaling O(n log n). On average, random pivot selection ensures balanced splits in expectation, also giving O(n log n). The worst case is $O(n^2)$ and occurs when every pivot produces a maximally unbalanced split, which I observed experimentally with repeated values (discussed in Section 4).

Heapsort: Building the max-heap takes O(n) time because lower levels of the tree require less sifting work. Each of the n-1 extractions then calls heapify at O(log n) cost, giving O(n log n) total. Crucially, Heapsort's worst case is also O(n log n), making it more predictable than Quicksort.

Space complexity: Randomized Quicksort uses O(log n) auxiliary space on average for the call stack (O(n) worst case). Heapsort uses O(log n) auxiliary space from recursive heapify calls, which is effectively O(1) since it sorts in place.

3. Performance vs Complexity

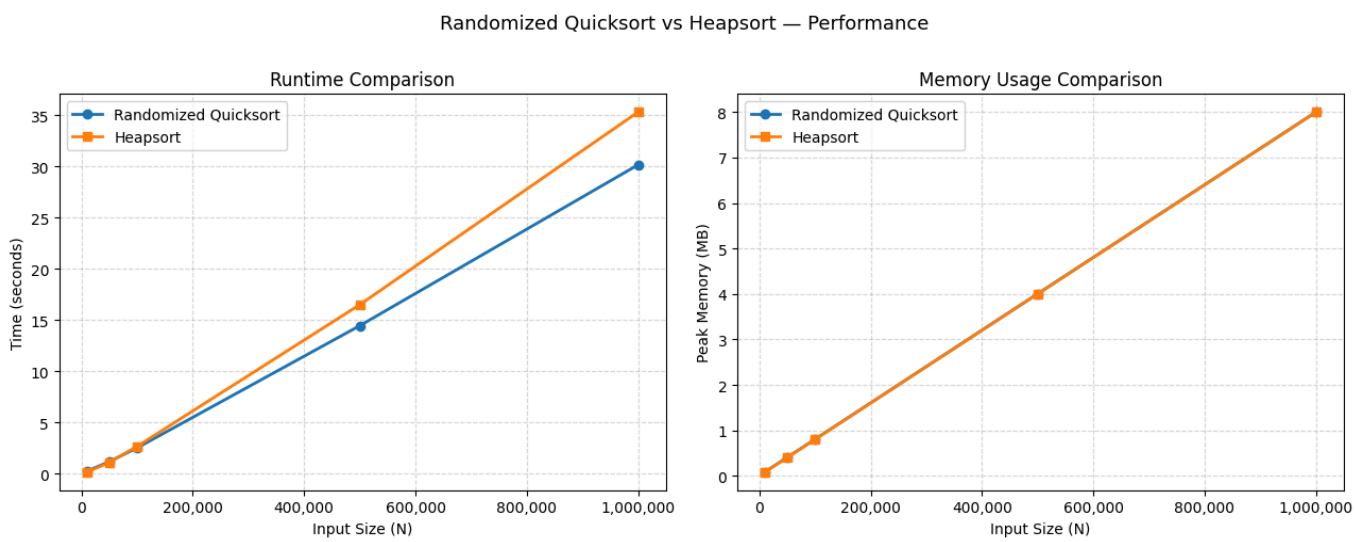
The table below shows measured runtimes align well with the expected O(n log n) behavior for both algorithms across random inputs.

N	QS Time (s)	QS Mem (MB)	HS Time (s)	HS Mem (MB)	Notes
10,000	0.2614	0.086	0.1607	0.081	ok
50,000	1.1858	0.405	1.1196	0.401	ok
100,000	2.5286	0.804	2.6752	0.801	ok
500,000	14.4493	4.005	16.4985	4.002	ok
1,000,000	30.1441	8.004	35.2940	8.002	ok

To verify O(n log n) growth, I can check whether scaling N by 10x scales the runtime by roughly 10 * log(10) ≈ 33x. From N = 100,000 to N = 1,000,000, Quicksort goes from 2.53s to 30.14s (roughly 12x) and Heapsort from 2.68s to 35.29s (roughly 13x). This is consistent with O(n log n) scaling. These times are higher than expected for a compiled language because Python's interpreted overhead adds a constant multiplier, but the growth curve matches theory.

No RecursionError or MemoryError occurred during any benchmark run.

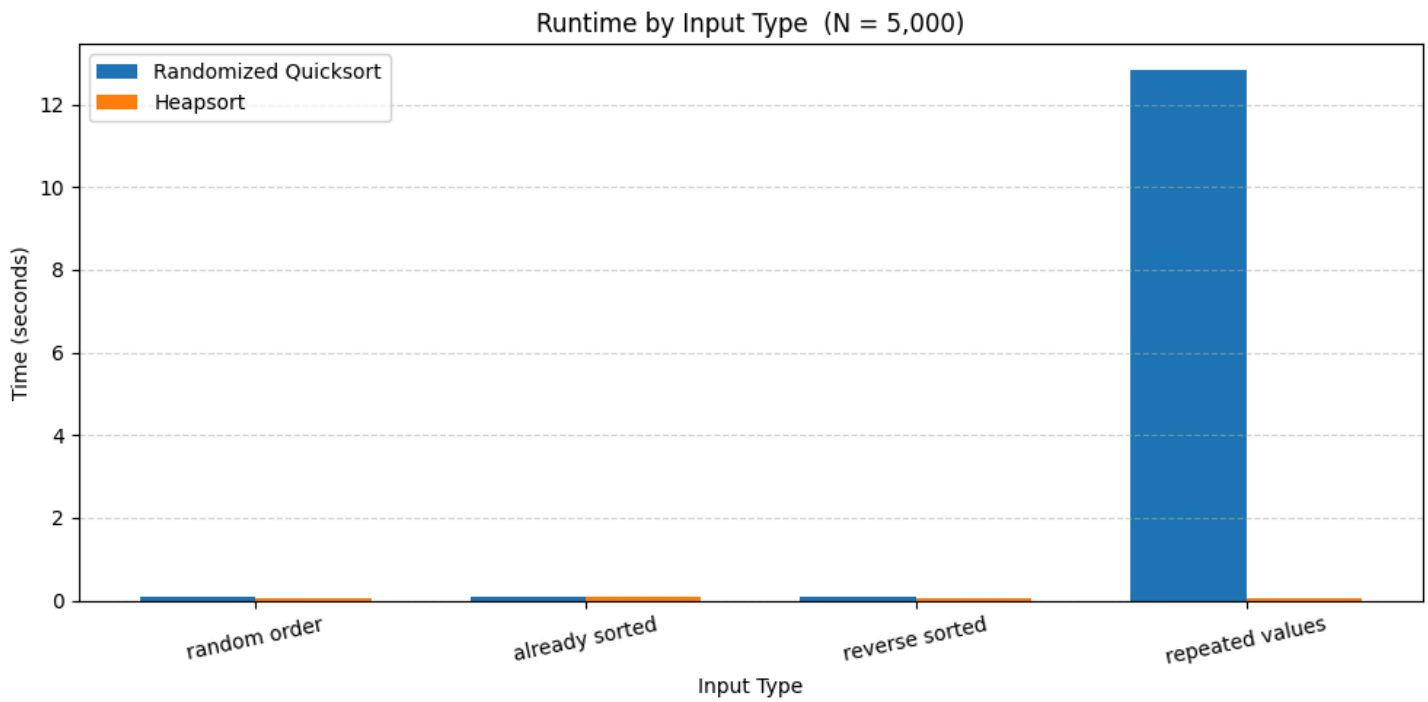
4. Performance Comparison



Both algorithms perform comparably on random inputs across all sizes. At N = 1,000,000, Quicksort takes 30.14s and Heapsort takes 35.29s. Quicksort is marginally faster here, likely due to better cache locality from its sequential partition scans compared to Heapsort's less predictable index jumps during heapify. Memory usage grows linearly with N for both algorithms and is nearly identical at every size tested (8.004 MB vs 8.002 MB at N = 1,000,000). This is expected since both sort in place and the dominant memory cost is the input array itself.

Input Type Effect on Performance (N = 5,000)

Input Type	QS Time (s)	QS Mem (MB)	HS Time (s)	HS Mem (MB)
Random order	0.1019	0.043	0.0646	0.044
Already sorted	0.0859	0.042	0.0804	0.043
Reverse sorted	0.0915	0.045	0.0695	0.041
Repeated values	12.8171	0.106	0.0490	0.041



---

5. Observations and Discussion

Observation1:

I did not encounter any RecursionError or MemoryError during the benchmark runs at N = 10,000 through N = 1,000,000. This is because I set the recursion limit to 200,000 at the start of the script. Without this, Python's default limit of 1,000 would cause a RecursionError for Quicksort at any moderately large input.

Observation2:

The most significant finding was Quicksort's behavior on repeated values. At N = 5,000, Quicksort took 12.82 seconds while Heapsort completed in 0.049 seconds, a difference of roughly 260x. This is a direct consequence of how the Lomuto partition scheme handles equal elements. When the array has many duplicates, even a randomly chosen pivot frequently equals most of the remaining elements. The partition still places all equal elements on one side, producing a subarray of near-n size and a near-zero-size subarray every time. This degenerates to $O(n^2)$ in practice and is also visible in the memory spike: Quicksort consumes 0.106 MB on repeated values vs 0.041-0.045 MB on all other types, due to the deep call stack from unbalanced recursion.

Heapsort is completely unaffected by repeated values because it never makes comparisons relative to a pivot. The heapify operation is purely structural and index-driven, so element distribution has no impact on its branching behavior.

Observation3:

I expected these to be problematic for Quicksort, but randomized pivot selection eliminates this concern entirely. At N = 5,000, Quicksort on already-sorted input takes 0.086s and on reverse-sorted takes 0.092s, both comparable to random order (0.102s). This confirms that randomization successfully avoids the classical $O(n^2)$ worst case that would occur if the last element was always used as the pivot.

Summary

Property	Randomized Quicksort	Heapsort
Sensitive to input type?	Yes - degenerates on repeated values	No - consistent across all types
Memory usage stable?	No - spikes with deep recursion	Yes - always O(1) auxiliary
Best observed input type	Already sorted (randomized pivot helps)	Reverse sorted (marginally)
Worst observed input type	Repeated values ($O(n^2)$ behavior)	Already sorted (marginal)

II) Write up

1. Randomized Quicksort tracing

Best Case - Pivot always splits array into two equal halves

Input: [3.0, 7.0, 1.0, 5.0, 9.0, 2.0, 8.0, 4.0]

In the best case I always select the median element as pivot, which produces perfectly balanced partitions at each step.

```
step 1: pivot=5.0  array=[3.0, 1.0, 4.0, 2.0, 5.0, 7.0, 8.0, 9.0]  pivot at index 4
step 2: pivot=3.0  array=[2.0, 1.0, 3.0, 4.0, 5.0, 7.0, 8.0, 9.0]  pivot at index 2
step 3: pivot=2.0  array=[1.0, 2.0, 3.0, 4.0, 5.0, 7.0, 8.0, 9.0]  pivot at index 1
step 4: pivot=8.0  array=[1.0, 2.0, 3.0, 4.0, 5.0, 7.0, 8.0, 9.0]  pivot at index 6
Final output: [1.0, 2.0, 3.0, 4.0, 5.0, 7.0, 8.0, 9.0]
```

Only 4 partition steps for 8 elements, matching the expected O(log n) recursion depth of the best case. Each pivot lands near the center, producing balanced halves.

Worst Case - Sorted input, pivot always chosen from end

Input: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]

In the worst case the last element is always the pivot. On a sorted array this means the pivot is always the largest, so all other elements go left and nothing goes right.

```
step 1: pivot=8.0  array=[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]  pivot at index 7
step 2: pivot=7.0  array=[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]  pivot at index 6
step 3: pivot=6.0  array=[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]  pivot at index 5
step 4: pivot=5.0  array=[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]  pivot at index 4
step 5: pivot=4.0  array=[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]  pivot at index 3
step 6: pivot=3.0  array=[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]  pivot at index 2
step 7: pivot=2.0  array=[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]  pivot at index 1
Final output: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]
```

7 partition steps for 8 elements, showing O(n) recursion depth which confirms $O(n^2)$ total work. Each step places the pivot at the far right of the remaining subarray, leaving a subproblem of size n-1 every time.

Average Case - Random input with randomized pivot selection

Input: [32.7, 15.8, 64.8, 8.1, 53.5, 36.8, 6.7, 50.7]

step 1: pivot=32.7 array=[15.8, 8.1, 6.7, 32.7, 53.5, 36.8, 64.8, 50.7] pivot at index 3
step 2: pivot=15.8 array=[6.7, 8.1, 15.8, 32.7, 53.5, 36.8, 64.8, 50.7] pivot at index 2
step 3: pivot=8.1 array=[6.7, 8.1, 15.8, 32.7, 53.5, 36.8, 64.8, 50.7] pivot at index 1
step 4: pivot=50.7 array=[6.7, 8.1, 15.8, 32.7, 36.8, 50.7, 64.8, 53.5] pivot at index 5
step 5: pivot=64.8 array=[6.7, 8.1, 15.8, 32.7, 36.8, 50.7, 53.5, 64.8] pivot at index 7
Final output: [6.7, 8.1, 15.8, 32.7, 36.8, 50.7, 53.5, 64.8]

5 partition steps for 8 elements. The splits are neither perfectly balanced (best) nor fully skewed (worst), typical of a random input. The recursion depth is O(log n) in expectation, giving O(n log n) average-case behavior.

---

2. Randomized Quicksort

Best case - pivot always splits into two equal halves:

$T(n) = 2T(n/2) + O(n)$
$T(1) = O(1)$

Worst case - pivot always lands at one end:

$T(n) = T(n-1) + T(0) + O(n) = T(n-1) + O(n)$
$T(1) = O(1)$

Average case - pivot is a random rank, expected split is n/2 : n/2:

$T(n) = (1/n) * \Sigma [T(k) + T(n-k-1)] + O(n)$ for $k = 0$ to $n-1$
$T(1) = O(1)$

The average case recurrence averages over all possible pivot positions. It can be shown using substitution that this solves to $T(n) = O(n \log n)$.

---

3. Recursion Tree

Recursion Tree - best case $T(n) = 2T(n/2) + cn$

```
Level 0:            cn                          = cn
                  /    \
Level 1:        cn/2      cn/2                  = cn
              /   \    /   \
Level 2:     cn/4  cn/4  cn/4  cn/4              = cn
              ...
Level log n:  (n leaves, each O(1))              = cn
```

There are log n + 1 levels, each contributing cn work, giving cn * log n = O(n log n) total.

Solving the Recurrence

Best and average case - $T(n) = 2T(n/2) + cn$:

By Master Theorem (Case 2):

a = 2, b = 2, f(n) = cn = O(n)
log_b(a) = log_2(2) = 1
f(n) = O(n^1) = O(n^log_b(a)) → Case 2 applies
T(n) = O(n log n)

Worst case - T(n) = T(n-1) + cn:

T(n) = T(n-1) + cn
= T(n-2) + c(n-1) + cn
= ...
= c(1 + 2 + ... + n)
= $O(n^2)$

---

4. Heapsort Tracing

Build-Max-Heap Phase

Input: [4.0, 10.0, 3.0, 5.0, 1.0, 7.0, 8.0, 2.0]

Last non-leaf index = 3. I start heapify from index 3 downward to index 0.

heapify(3): value=5.0, children=2.0 → no swap needed
array: [4.0, 10.0, 3.0, 5.0, 1.0, 7.0, 8.0, 2.0]

heapify(2): value=3.0, children=7.0, 8.0 → swap arr[2]=3.0 with arr[6]=8.0
array: [4.0, 10.0, 8.0, 5.0, 1.0, 7.0, 3.0, 2.0]

heapify(1): value=10.0, children=5.0, 1.0 → no swap needed
array: [4.0, 10.0, 8.0, 5.0, 1.0, 7.0, 3.0, 2.0]

heapify(0): value=4.0, children=10.0, 8.0 → swap arr[0]=4.0 with arr[1]=10.0
then value=4.0, children=5.0, 1.0 → swap arr[1]=4.0 with arr[3]=5.0
array: [10.0, 5.0, 8.0, 4.0, 1.0, 7.0, 3.0, 2.0]

Max-heap built: [10.0, 5.0, 8.0, 4.0, 1.0, 7.0, 3.0, 2.0]

Full Heapsort Pass - Extract-Max Loop

Starting heap: [10.0, 5.0, 8.0, 4.0, 1.0, 7.0, 3.0, 2.0]

step 1: swap root=10.0 with arr[7]=2.0
heap: [2.0, 5.0, 8.0, 4.0, 1.0, 7.0, 3.0] sorted: [10.0]
heapify: 2.0 → 8.0 → 7.0
heap: [8.0, 5.0, 7.0, 4.0, 1.0, 2.0, 3.0] sorted: [10.0]

step 2: swap root=8.0 with arr[6]=3.0
heap: [3.0, 5.0, 7.0, 4.0, 1.0, 2.0] sorted: [8.0, 10.0]
heapify: 3.0 → 7.0
heap: [7.0, 5.0, 3.0, 4.0, 1.0, 2.0] sorted: [8.0, 10.0]

step 3: swap root=7.0 with arr[5]=2.0
heap: [2.0, 5.0, 3.0, 4.0, 1.0] sorted: [7.0, 8.0, 10.0]
heapify: 2.0 → 5.0 → 4.0
heap: [5.0, 4.0, 3.0, 2.0, 1.0] sorted: [7.0, 8.0, 10.0]

step 4: swap root=5.0 with arr[4]=1.0
heap: [1.0, 4.0, 3.0, 2.0] sorted: [5.0, 7.0, 8.0, 10.0]
heapify: 1.0 → 4.0 → 2.0
heap: [4.0, 2.0, 3.0, 1.0] sorted: [5.0, 7.0, 8.0, 10.0]

step 5: swap root=4.0 with arr[3]=1.0
heap: [1.0, 2.0, 3.0] sorted: [4.0, 5.0, 7.0, 8.0, 10.0]
heapify: 1.0 → 3.0
heap: [3.0, 2.0, 1.0] sorted: [4.0, 5.0, 7.0, 8.0, 10.0]

step 6: swap root=3.0 with arr[2]=1.0
heap: [1.0, 2.0] sorted: [3.0, 4.0, 5.0, 7.0, 8.0, 10.0]
heapify: 1.0 → 2.0
heap: [2.0, 1.0] sorted: [3.0, 4.0, 5.0, 7.0, 8.0, 10.0]

step 7: swap root=2.0 with arr[1]=1.0
heap: [1.0] sorted: [2.0, 3.0, 4.0, 5.0, 7.0, 8.0, 10.0]
single element, no heapify needed

Final sorted output: [1.0, 2.0, 3.0, 4.0, 5.0, 7.0, 8.0, 10.0]

Each extraction places the current maximum into its final sorted position at the end of the array. After 7 extractions (n-1 steps), the entire array is sorted in ascending order.

III) AI Usage Statement

1. I used Perplexity AI to help me read and understand the core logic behind randomised Quicksort and Heapsort before writing any code. Specifically, I asked it to explain how the Lomuto partition scheme works, how randomized pivot selection prevents worst-case behavior, and how the heapify operation maintains the max-heap property during the build and extract phases.

2. I also used Perplexity AI for help writing the matplotlib code used to generate the performance plots in report_benchmark.py and report_input_types.py. This covers the runtime vs input size line charts saved as performance_plot.png and the input type bar chart saved as input_type_plot.png.

3. I used Grammarly to spell-check the written sections of this report. I also used Microsoft Edge co-pilot in helping me format this report in .docx.